NLP HW1

Homework 1 Answers - Harihar Prasad Deepak (hdeepak1)

Part 1

1.4:

1. Ten random sentences are as follows:
   i. a delicious fine president on every sandwich on a president wanted a floor !
   ii. a sandwich under the president pickled every pickle !
   iii. a chief of staff kissed the pickle .
   iv. a chief of staff in every president in a sandwich on every pickle understood a sandwich on the chief of staff on
   every pickle under every president under the pickle under the pickle on the sandwich under the chief of staff on the sandwich under
   every chief of staff in every floor in every sandwich under every floor in every pickle in a sandwich on a sandwich on the sandwich
   under a fine sandwich with every pickle under a president with every chief of staff with a president with a fine pickle in every chief
   of staff on a president under every floor with the chief of staff on the sandwich on every pickle in every pickle under every chief of
   staff with the pickle on the chief of staff with the fine president on the pickle with every floor with a floor on a floor under the
   president in the pickle under a floor in the president in a delicious chief of staff on a floor with the fine pickle under every chief
   of staff on every chief of staff with a pickled floor with every perplexed fine floor with a pickle on the chief of staff under the
   sandwich on every president under the chief of staff with every president on every sandwich on a president under a chief of staff under a pickle
   in a floor !
   v. is it true that the pickle ate a sandwich in a floor on the pickle on the chief of staff with the floor under the floor ?
   vi. is it true that a president under a pickle under a president in the chief of staff in a pickle in a chief of staff with the sandwich with the pickle
   under the floor with the sandwich with every sandwich on a fine president in every chief of staff with a pickle pickled the sandwich ?
   vii. a sandwich kissed a pickle .
   viii. the pickle understood a sandwich in every chief of staff !
   ix. the sandwich under the president in a president on a chief of staff in a president with a fine delicious perplexed floor on a chief of staff with
   a chief of staff under the sandwich in the president in the fine president understood a floor .

x. the sandwich under a sandwich kissed every pickle on a floor with the president in a chief of staff under the chief of staff on a president under a
chief of staff with the pickle with the pickle in the chief of staff with a sandwich with a floor on the floor in the sandwich on a chief of staff in
every perplexed chief of staff on every fine sandwich under every sandwich with the delicious delicious sandwich under the delicious president with a
floor in a sandwich on the pickled pickle in every floor under every floor on every pickle on a floor under the perplexed delicious delicious pickle
in a president with the president with a floor in a chief of staff under every pickle .

2. 2 random sentences generated using my script, with --tree:
i. (ROOT (S (NP (NP (Det the)
(Noun pickle))
(PP (Prep with)
(NP (Det every)
(Noun chief
of
staff))))
(VP (Verb understood)
(NP (NP (Det every)
(Noun pickle))
(PP (Prep in)
(NP (NP (NP (NP (Det every)
(Noun floor))
(PP (Prep in)
(NP (NP (NP (Det every)
(Noun pickle))
(PP (Prep with)
(NP (Det a)
(Noun chief
of
staff))))
(PP (Prep in)
(NP (Det a)
(Noun (Adj delicious)
(Noun chief
of
staff)))))))
(PP (Prep with)
(NP (Det the)
(Noun sandwich))))
(PP (Prep with)
(NP (Det a)
(Noun president))))))))

.)
ii. (ROOT is
it
true
that
(S (NP (Det every)
(Noun sandwich))
(VP (Verb pickled)
(NP (Det the)
(Noun (Adj delicious)
(Noun (Adj pickled)
(Noun pickle))))))
?)

3. 2 generations with --tree, but with max_expansions set to 5:
   i. (ROOT (S (NP (NP (Det the)
   ...)
   ...)
   ...)
   !)
   ii. (ROOT (S (NP (Det a)
   (Noun president))
   ...)
   !)

Part 2

2.1:

1. The program generates so many long sentences primarily because of these two rules:
   NP -> NP PP
   PP -> Prep NP.
   They're special because this definition is NP is heavily recursive (calling NP returns NP and PP, and calling PP returns NP again).
2. In our grammar, consecutive adjectives only occur if "Noun" calls "Adj Noun" twice consecutively - that is:
   (Noun (Adj (Noun (Adj Noun)))). However, since we have five other terminal nouns (president, sandwich etc) the probability of "Noun" resolving to
   "Adj Noun" a single time is 1/6, and the odds of it happening twice in a row are even lower, at 1/36. That's why we don't see consecutive adjectives
   as often.
3. To make sentences shorter, you could either upweight the probability that NP resolves to (Det Noun) from 1 to a higher number, or you could downweight the

probability that NP resolves to (NP PP). The two are functionally the same.

To make adjectives more frequent, you upweight the probability that Noun resolves to (Adj Noun).

4. After experimenting, apart from finetuning the probability that NP resolves to (NP PP) and the probability that Noun resolves to (Adj Noun), the changes that I found resulted in more understandable sentences were primarily lexical. I reranked the determiners, making (the, a, every) (3, 2, 1) respectively, to better reflect their frequency in the english language. I also downranked the verb "pickled" to 0.2 from 1, since it occurs much less frequently than the rest of the verbs in the context of the nouns in the grammar.
5. Ten random sentences from grammar2.gr are as follows:
    1. is it true that a pickle ate a pickle in the president under every president in every sandwich in the chief of staff ?
    2. a chief of staff in the fine sandwich under the chief of staff with the sandwich with the pickle with the perplexed pickled fine sandwich on the delicious chief of staff wanted the delicious pickle !
    3. the pickled floor with the delicious chief of staff with the pickled fine perplexed sandwich understood a fine floor !
    4. is it true that a fine pickle in the chief of staff in the pickled perplexed pickle wanted the floor with every pickled fine chief of staff with the fine pickle in every sandwich under a perplexed pickle ?
    5. a pickle ate the perplexed fine sandwich under the delicious pickle .
    6. the delicious perplexed fine chief of staff ate the floor .
    7. a pickle understood a pickle .
    8. a pickled perplexed perplexed pickle pickled a pickle !
    9. the pickle wanted every perplexed sandwich in a fine pickle .
    10. is it true that the fine picklead pickled president kissed the chief of staff ?

2.3:

9. First I added all the new words to the vocabulary. Then I split "Nouns" into Proper Nouns (PropNoun) and Common Nouns (ComNoun), and added NP -> Det ComNoun and NP -> PropNoun to disallow phrases like "every Sarah" or "the Sarah". Then I added the rule NP -> NP Conj NP to allow for phrases like "Sarah and the president", along with the rule VP -> CoordVerb Conj VP to allow sentences like "wanted and ate a sandwich", and made a new Coordverb category to disallow sentences like "The president perplexed and sighed." in conjuction with the Intransitive Verb rule. Then I added "sighed" into a new Intransitive Verb (InVerb) category, and added the rule VP -> InVerb, to allow S -> NP VP structure to generate sentences like "The

president sighed.". Then I added special rules to allow construction of sentences using "that" by introducing a Complementizer Phrase (CP) such that CP -> Comp S, then added a new Transitive Verb (TransVerb) category for verbs to operate on these
Complementizer Phrases. Then I added the capability for many adverbs with the Adj -> Adv Adj rule, and Adv -> very. Finally, I added a Prepositional Verb (PrepVerb) category for verbs that allow a preposition after them, and then added a VP -> PrepVerb PP rule to allow sentences like "The president worked on the table."

10. Ten random sentences from grammar3.gr are as follows:
    1. is it true that every pickled delicious very very pickled very perplexed delicious proposal understood and sighed ?
    2. Sally under Sally and the sandwich and Sally under a delicious desk and the perplexed proposal and every floor and Sally and Sally and the chief of staff with a floor under Sally and Sally on Sally and Sally with the pickle in every delicious very delicious perplexed proposal in the pickled fine proposal and Sally on a pickled desk on the proposal with the perplexed sandwich and a pickled delicious president and every proposal and Sally under Sally sighed !
    3. it perplexed Sally that every floor and every desk perplexed every chief of staff.
    4. the pickled desk wanted and sighed .
    5. is it true that Sally and a very perplexed pickled floor and Sally under every perplexed delicious fine pickled very perplexed sandwich on every chief of staff and every very delicious president and Sally and a very delicious very very pickled pickled pickled very perplexed floor and the floor and Sally and Sally under Sally and the desk under a proposal and the pickle and Sally and Sally and the very delicious proposal and Sally with the sandwich and Sally and Sally under the sandwich on Sally and Sally on Sally and every desk in every floor and Sally and a chief of staff under a proposal under a proposal and Sally and Sally and every delicious pickle and the delicious pickled sandwich and a perplexed president and Sally and Sally and Sally and a perplexed proposal and Sally and Sally worked and kissed a pickle and every pickle and every perplexed desk and the delicious sandwich in Sally and the pickle and every sandwich on every floor ?
    6. is it true that Sally in Sally with Sally and the pickle with Sally and Sally with the desk and the proposal and the perplexed floor and Sally with a fine pickled perplexed delicious president and Sally on Sally with Sally and Sally and the chief of staff with a floor and Sally with Sally with the pickled pickle in Sally under the floor and Sally with Sally and the pickle and the fine fine pickled president and Sally and Sally and a pickle and Sally and Sally with Sally under Sally and Sally and Sally on every perplexed president under a pickle in Sally under Sally on the very fine chief of staff and a fine floor and the very very delicious chief of staff under Sally and Sally and Sally and Sally and a sandwich and a chief of staff and every chief of staff in Sally on Sally and the pickle and the pickle and Sally and Sally and Sally and the delicious delicious sandwich

and the president on Sally with Sally in Sally in the pickle with the pickle and Sally with Sally in Sally and Sally on Sally and Sally and Sally and a very perplexed perplexed sandwich on Sally with a chief of staff and Sally in the president in Sally and the chief of staff on the very delicious pickle and every floor under Sally in Sally and Sally and .....?

7. Sally and the proposal and Sally kissed a president .
8. Sally understood and worked in Sally .
9. Sally understood and understood and wanted every delicious chief of staff !
10. every pickled sandwich understood that that Sally sighed perplexed every chief of staff !

Part 3

3.1:

1. (a) The other derivation is:
   (ROOT
   (S
   (NP
   (NP
   (Det every)
   (Noun sandwich))
   (PP
   (Prep with)
   (NP
   (NP
   (Det a)
   (Noun pickle))
   (PP
   (Prep on)
   (NP
   (Det the)
   (Noun floor))))))
   (VP
   (Verb wanted)
   (NP
   (Det a)
   (Noun president))))
   .)

(b) Yes, the derivation does matter. In the original formulation, "on the floor" was a prepositional phrase attached to "every sandwich with a pickle", which

means that the sandwich as a whole, with the pickle, was on the floor. In our new formulation, on the floor is attached to "a pickle", which now means that every sandwich has a pickle that was on the floor, but the sandwich itself was never on the floor, and it is this sandwich that has a pickle that was on the floor that wants a president.

2. Yes, the parser does misunderstand the original derivation that was "intended" by randsent on grammar3. The two main rules responsible for this are NP -> NP Conj NP and NP -> NP PP. For example, "Sally and the president and a sandwich" can be grouped as either "[Sally and the president] and a sandwich" or "Sally and [the president and a sandwich]", which is an example of how NP -> NP Conj NP results in ambiguity. Likewise, "the sandwich with a pickle on the floor" can attach the PP "on the floor" to the NP "a pickle" or to the larger NP "the sandwich with a pickle".
3. There are five ways to analyze the given noun phrase under grammar.gr. We have 4 smaller NPs here, which we label as follows:
   A = every sandwich
   B = a pickle
   C = the floor
   D = the chief of staff

   With these labels, the five possible structures are:
   1. (((A with B) on C) under D)

   ```
   2. ((A with (B on C)) under D)

   3. ((A with B) on (C under D))

   4. (A with ((B on C) under D))

   5. (A with (B on (C under D)))
   ```

   The reason there are 5 is that each PP contains another NP, and that NP can itself receive another PP using NP -> NP PP. With three PPs, there are five distinct ways of arranging those recursive attachments.

   ./parse corroborates this and says that there are 5 possible parses.
4. Yes, there are certain patterns for which formulations result in more possible parses. For grammar.gr, ambiguity mainly comes from recursive rules like NP -> NP PP and PP -> Prep NP. You can isolate the cause to these patterns by observing a sentence like "every president understood a sandwich", which has no PP chains and has 1 parse. Sentences with many PPs have more parses: “a floor pickled the pickle

under a president in the chief of staff” has 2 parses, and “the pickle on a president on the president under every pickle kissed every sandwich” has 5. The more PPs there ae, the more possible parses there are because you can attach different PPs to
different NPs.

grammar3.gr also has the above PP ambiguity in parsing, but conjugations add another source of ambiguity through the rule NP -> NP Conj NP. For example,
“the chief of staff and the pickled pickled sandwich and Sally with Sally under a proposal thought and worked on Sally” has 14 parses, with a major source
of ambiguity being how to join the different NPs with conjugations. Longer sentences combining many conjunctions and PPs had tens of thousands or even
millions of parses.

Here's one from grammar3 that has 987,369,656 parses:
“that it perplexed Sally and Sally and Sally on Sally and Sally on Sally on every perplexed fine sandwich with the delicious chief of staff and the delicious
desk with the fine pickled perplexed floor and the perplexed very delicious pickle and Sally that every chief of staff on the floor and the desk and Sally and
Sally and Sally under the perplexed proposal and Sally and a fine very fine perplexed delicious floor under Sally and Sally worked in Sally perplexed Sally !”

5. (a) p(best_parse) is so small because it calculates the probability that a random greenfield generation would generate the given parse tree. In this case, you multiply the probability of choosing each word or formulation (probability of that word or formulation/probability of the whole sample space) since they're each independent, and you end up with 0.00005144. Here, ROOT -> S . has probability 1/3, because ROOT has 3 equally weighted rules, S -> NP VP has probability 1, because it is the only S rule, for the first NP, NP -> Det Noun has probability 1/2, because NP can either become Det Noun or NP PP, Det -> the has probability 1/3, Noun -> president has probability 1/6, VP -> Verb NP has probability 1, Verb -> ate has probability 1/5, for the second NP, again NP -> Det Noun is 1/2,
   Det -> the is 1/3 and Noun -> sandwich is 1/6. Multiplying this all together: (1/3) *1* (1/2) *(1/3)* (1/6) *1* (1/5) *(1/2)* (1/3) * (1/6) = 1/19440 ~ 0.00005144.

p(sentence) is equal to p(best_parse) because there's only one possible way of parsing the given sentence, and thus saying "There is X probability of generating
this particular parse of the sentences and saying "There is X probability of generating this sentence" is the same thing.

p(best_parse | sentence) = 1, again, because there's only one possible way of parsing the given sentence, and thus given the sentence the probability that its

tree is the best parse is 100%, or 1.

(b) Here, p(best_parse | sentence) = 0.5 because there's 2 different ways of parsing the sentence that are both equally probable - ie, multiplying the probability of choosing each word or formulation at each step gives you the same probability for both the parse trees.

(c) Information theory defines the "surprise" of an event, which we encode in bits, as −log2p. Cross entropy is basically the average surprise of each word in the corpus. Thus, cross-entropy = 2.435 bits = -(-43.833 log-prob. / 18 words) can be decomposed as follows: -43.833 log-probability adds the base-2 log probabilities of the two sentences to get total surprise: log2(5.144e-05) + log2(1.240e-09) = -43.833. To average this surprise, we divide by the total number of words, which is 18, which gives us -2.435. Then since surprise is -log2p, we negate the entire thing again to get 2.435, which is the average number of bits of each word in the corpus, which is how "surprising" each word is based on the probability of it occuring.

(d) Perplexity is just cross-entropy taken out of the log, which in this case is 2^2.435 ~ 5.4.

(e) The grammar assigns the president ate . probability 0 because VP -> Verb NP requires an object after every verb. Since -log2(0)= infinity, any corpus containing that sentence has infinite cross-entropy, meaning the compression program can't compress this corpus at all.

6. (a) The average entropy of grammar2 in bits per word is 2.204. This is the command I used to compute it:

```
python3 randsent.py -n 10000 -g grammar2.gr
| grep -v '...'
| ./parse -P -g grammar2.gr
| grep "cross-entropy"
| awk '$4 != "inf" {sum += $4; n++} END {print sum / n}'
```

The pipe generates 10,000 random sentences from grammar2.gr, removes any truncated ones containing ..., parses the remaining sentences with the same grammar, keeps only the cross-entropy outputs, ignores any infinite values, and averages the remaining cross-entropies.

(b) grammar3 has a higher entropy than grammar2 - 2.255 compared to 2.204, calculated over 10,000 samples. The entropy probably increases due to the increased

vocabulary and the new clause structures, but doesn't get too much bigger since the new structures are not quite random and most of the variability comes from NP and PP structures that were already present in grammar2.

(c) The original grammar is difficult to estimate entropy for by sampling because NP -> NP PP and PP -> Prep NP create a recursive loop that can produce extremely long noun phrases. Some generated sentences hit the maximum-expansion limit and contain ..., which are not valid sentences in the grammar, so the parser assigns them probability 0 and the cross-entropy becomes infinite or undefined. Grammar2 reduces this problem by lowering the weight of NP -> NP PP, making runaway recursion much less common, and since Grammar3 is adapted from Grammar2 the fix carries over.

7. Computing the numbers gives this:

grammar2 entropy on grammar2 corpus: 2.208 bits/word
grammar entropy on grammar2 corpus: 2.392 bits/word
grammar3 entropy on grammar2 corpus: 2.988 bits/word

So yes, our assumption that grammar2 predicts the grammar2 corpus better holds true. We also see that grammar predicts the grammar2 corpus better than grammar3 - this is probably because grammar2 is considerably closer to grammar than grammar3. More similar rules, not as much extra vocabulary, no conjuctions, etc.

Part 4

4.1. The two changes I chose to implement were (a) and (b). For (a), I first introduced three ComNouns that start with vowels (apple, orange, umbrella) as VowComNoun and two Adjs that start with a vowel (amazing, ambivalent) as VowAdj. I then split determiners into Det (every, the), which works with both vowel and consonant starting phrases, VowDet (an), which works only with phrases starting with a vowel, and NoVowDet(a) which works only with phrases that don't start with a vowel. I then renamed normal ComNouns and Adjs (those that don't start with vowels) to NoVowComNoun and NoVowAdj respectively.

Then I introduced the following rules (comments only in this version):

1 NP Det ComNoun # the, every can work with every common noun
1 NP NoVowDet NonVowPhrase # a can only work with phrases that don't start with vowels
1 NP VowDet VowPhrase # an can only work with phrases that start with vowels

```
                                        # non vowel phrases can be:
```

1 NonVowPhrase NoVowComNoun # a common noun that doesn't start with a vowel (eg president)
1 NonVowPhrase NoVowAdj ComNoun # an adjective that doesn't start with a vowel followed by any common noun (eg fine apple, perplexed president)
1 NonVowPhrase AdvPhrase ComNoun # our only adverb (very) followed by anything - we use advphrase to ensure that we get an adverb, and to leave possibility
for recursive verys (eg very very very ambivalent apple)

```
                                        # vowel phrases can be:
```

1 VowPhrase VowAdj ComNoun # an adjective starting with a vowel followed by a common noun (eg amazing president, ambivalent very perplexed floor)
1 VowPhrase VowComNoun # a common noun starting with a vowel (eg umbrella)

2 ComNoun Adj ComNoun # this section just lets ComNoun decompose into either NoVowComNoun or VowComNoun
1 ComNoun NoVowComNoun
1 ComNoun VowComNoun

0.7 Adj Adv Adj # this section lets adjectives decompose into either NoVowAdj or VowAdj
1 Adj NoVowAdj
1 Adj VowAdj

1 AdvPhrase Adv Adj # allows for 100% probability of very + recursive formulations (very very very)

For (b), I first introduced the new question component, Q, and added a new ROOT formulation of "1 ROOT Q ?". I then basically had to create a parallel formulation for Q for almost all of the S rules by adding an AuxVerb at the start (did or will) followed by replacing by normal Verb Phrases with Question Verb Phrases that brought ALL verbs that were not in a Comparative Phrase into the present tense: the rules are as follows:

4 Q AuxVerb NP QVP
1 Q AuxVerb Expl PresTransVerb NP CP
1 Q AuxVerb CP PresTransVerb NP

1 QVP PresVerb NP
1 QVP PresCoordVerb Conj QVP
1 QVP PresInVerb
1 QVP PresPrepVerb PP
1 QVP PresVerbC CP

Then I just had to go in and change the vocabulary itself to turn ate -> eat, thought -> think, etc etc for all verbs to make them appropriate to ask questions.

Note: This homework assignment was so, so, so, so long. It took me ~15 hours to complete. Very fun though!